

베타테스트 자동화 도구 개발 계획서

SIZL Agentic Brain 베타테스트 운영을 위한
이슈 수집 · PR 자동 생성 · Google Sheet 대시보드

대상 저장소: Sizl-Neolab/SIZL-Agentic-Brain-Issue-Track

작성일: 2026 년 5 월 19 일

버전: 2.0 (Google Sheet 대시보드 구체화)

1. 개요

1.1 배경

SIZL Agentic Brain 서비스의 베타테스트 이슈는 GitHub 저장소 'Sizl-Neolab/SIZL-Agentic-Brain-Issue-Track'의 Issues 탭에 등록되고, 운영자가 Google Sheet 로 옮겨 수작업으로 관리하고 있다. 두 종류의 템플릿(이슈 리포트 / 개선 사항)이 운영 중이다.

이 방식의 운영상 한계는 다음과 같다.

- 일반 베타 테스터에게 GitHub 가입 및 Markdown 작성이 진입 장벽이다.
- GitHub Issue → Google Sheet 옮겨 적기에 사람 손이 매번 필요하다.
- 이슈 등록 → 원인 분석 → PR 작성 → 머지 → 보고로 이어지는 흐름이 수동 작업에 의존한다.
- 이슈와 PR 의 처리 현황이 한 곳에 집계되지 않아, 매일 별도 정리 작업이 필요하다.

1.2 목적

본 계획서는 (1) 베타 테스터의 진입 장벽을 낮추고, (2) 이슈 접수부터 LLM 기반 PR 초안 생성, (3) 일일 상태 보고 및 대시보드까지를 자동화하는 도구를 개발하기 위한 범위·일정·기술 선택을 정의한다.

1.3 핵심 워크플로우

1. 웹 화면으로 이슈 리포트 / 개선 사항 템플릿을 제공하여 베타 테스터로부터 입력을 받고, GitHub Issues 탭에 Open 상태로 자동 등록한다.
2. Open 된 이슈에 대해 지정된 LLM API 를 호출하여 분석 결과와 코드 수정 초안을 포함한 Pull Request 를 자동 생성한다.
3. 생성된 Open PR 은 개발자의 수동 검토와 머지 과정을 거쳐 해결한다(자동화 범위 외).
4. 매일 오후 5 시(Asia/Seoul)에 스케줄러가 이슈 및 PR 처리 상태를 Google Sheet 에 동기화하고, 대시보드 시트의 통계가 자동 갱신된다.

2. 시스템 구성

2.1 컴포넌트

- ① Web Frontend: 베타 테스터용 이슈/개선 입력 폼
- ② Backend API Server: 입력 검증, GitHub Issues API 호출, 이미지 업로드
- ③ LLM Worker: GitHub Webhook 을 받아 PR 초안을 자동 생성
- ④ Sheet Syncer: 이슈/PR 상태를 Google Sheet 로 동기화 (실시간 + 일일 17 시)

2.2 데이터 흐름

5. 베타 테스터가 웹 폼을 제출 → Backend 가 GitHub Issue 생성 → Issue 가 Open 상태로 등록
6. GitHub 가 issues.opened Webhook 을 LLM Worker 와 Sheet Syncer 로 동시 전송
7. Sheet Syncer 는 즉시 Raw Issues 시트에 새 행을 추가
8. LLM Worker 는 LLM API 로 분석/패치 초안을 만들고 새 브랜치에 커밋 후 PR 생성
9. PR 생성/상태변경 시 Sheet Syncer 가 해당 행의 자동 컬럼만 갱신
10. 개발자가 PR 을 수동 검토·머지 → 머지 시 이슈 자동 Close
11. 매일 17:00 에 전수 동기화 + Daily Snapshot 시트에 통계 1 행 추가 → Dashboard 차트 자동 갱신

3. 이슈 템플릿 (웹 폼 ↔ GitHub Issue)

기존 저장소의 두 템플릿(이슈 리포트 / 개선 사항)을 그대로 유지하고, 두 템플릿 공통으로 '테스트 환경' 항목 1 개만 추가한다. 노란색 배경 항목이 신규 추가 항목이다.

3.1 이슈 리포트 템플릿 항목

섹션	입력 항목	구분	입력 형식 / 비고
Summary	테스트 계정	기존	이메일 입력
Summary	접속 시간	기존	datetime-local (자동으로 KST 변환)
Summary	테스트 영역	기존	선택 (Research agent / Document agent / etc.)
Summary	원인 유형	기존	선택 (Backend / Prompting / Cache / etc.)
Summary	원인 상세	기존	단문 텍스트
Summary	심각도	기존	선택 (치명적 / 높음 / 중간 / 낮음). 우선순위 P1~P4 로 매핑
Summary	테스트 환경	★ 추가	OS / 브라우저(+버전) / 디바이스 / 네트워크
테스트 시나리오	세부 기능	기존	단문 텍스트
테스트 시나리오	시나리오 구체적 설명	기존	멀티라인 텍스트
재현 방법	재현 빈도	기존	선택 (항상 / 가끔 / 드물게)
재현 방법	재현 단계	기존	동적 번호 입력
예상/실제 동작	예상 동작 / 실제 동작	기존	각각 멀티라인 텍스트
문제 발생 화면	스크린샷/영상	기존	드래그&드롭. GitHub 에 업로드 후 Markdown 링크로 본문 삽입
입력값 / 출력값	입력 / 실제 출력 / 기대 출력	기존	3 개의 멀티라인 텍스트
추가 의견	자유 기재	기존	멀티라인 텍스트

3.2 개선 사항 템플릿 항목

섹션	입력 항목	구분	입력 형식 / 비고
Summary	테스트 계정	기존	이메일 입력
Summary	테스트 영역	기존	선택
Summary	페이지 위치	기존	경로 입력 (예: /agents/research)
Summary	우선순위	기존	선택 (높음 / 중간 / 낮음)
Summary	테스트 환경	★ 추가	OS / 브라우저(+버전) / 디바이스 / 네트워크
개선 요청 내용	개선이 필요한 기능 / 현재 동작 / 기대 동작	기존	각각 텍스트
이유 및 기대효과	자유 기재	기존	멀티라인 텍스트
참고 화면 / 추가 의견	스크린샷 + 자유 기재	기존	드래그&드롭, 멀티라인 텍스트

3.3 자동 부여 라벨

- 이슈 유형: bug 또는 enhancement
- 우선순위: priority: P1 / P2 / P3 / P4 (치명적→P1, 높음→P2, 중간→P3, 낮음→P4)
- 테스트 영역: area: research / area: document / area: chat 등
- 출처 식별: beta-feedback (모든 웹 폼 제출 이슈에 공통)
- 자동 처리 식별: auto-pr-pending → auto-pr-created 또는 auto-pr-failed

4. Google Sheet 설계 (대시보드 포함)

운영 중인 시트의 컬럼 구조와 운영자의 수작업 영역을 그대로 보존하면서, 자동화 컬럼을 뒤쪽에 추가하는 방식이다. 시트는 다음 3 개로 분리한다.

4.1 시트 구조 개요

시트명	용도	갱신 주기 / 방식
Raw Issues	이슈 원장	Webhook 으로 실시간 행 추가/갱신 + 매일 17 시 전수 동기화
Daily Snapshot	일별 통계 누적	매일 17 시에 1 행 추가 (덮어쓰지 않고 append)
Dashboard	KPI + 차트	Raw Issues / Daily Snapshot 을 데이터 소스로 한 차트. 시트 열 때마다 자동 계산

4.2 [Raw Issues] 시트 컬럼 명세

기존 시트의 컬럼 19 개는 그대로 유지하고, 자동화용 컬럼 10 개를 뒤(T~AC)에 추가한다. 운영자가 직접 채우는 수작업 컬럼(L, M, N, O, P, S)은 자동화 도구가 절대 덮어쓰지 않는다.

열	컬럼명	구분	작성 주체	내용 / 비고
A	등록자	기존	자동	이슈 작성자 이름 (폼 입력값 또는 GitHub 계정)
B	팀	기존	폼 입력	Neo Lab / 컨설팅 / Ax 추진 등. 폼에 드롭다운 추가
C	테스트 계정	기존	자동	폼에서 입력한 테스트 계정
D	테스트 영역	기존	자동	ETL / 채팅 / 에이전트 / 로그인 등
E	세부 기능	기존	자동	Data Examiner / 채팅 / 새 에이전트 생성 등
F	발생 화면	기존	자동	URL 경로 (/chat, /agents 등)
G	테스트 시나리오	기존	자동	폼 입력값
H	발생 증상	기존	자동	폼 입력값

열	컬럼명	구분	작성 주체	내용 / 비고
I	재현 여부	기존	자동	Yes / No (드롭다운 데이터 검증)
J	발생 빈도	기존	자동	항상 / 가끔 / 드물게 (드롭다운)
K	우선순위	기존	자동	P1 / P2 / P3 / P4 (드롭다운, 색상)
L	원인 유형	기존	★ 수작업	Backend / Prompting / Cache 등. 운영자가 분석 후 기입
M	원인 상세	기존	★ 수작업	원인의 구체적 내용
N	테스트 담당자	기존	★ 수작업	처리 담당자 배정
O	조치 내용	기존	★ 수작업	어떤 조치를 했는지
P	조치 예정일	기존	★ 수작업	운영자 입력
Q	처리 일자	기존	자동	PR 머지 시점 자동 기록
R	처리 상태	기존	자동	Open / 진행중 / 완료
S	배포	기존	★ 수작업	배포 여부 / 일자
T	등록일시	★ 추가	자동	GitHub Issue createdAt (KST)
U	Issue 번호	★ 추가	자동	GitHub Issue Number (#142 등). 시트의 PRIMARY KEY
V	Issue URL	★ 추가	자동	HYPERLINK 함수로 표기. 클릭 시 GitHub 로 이동
W	PR 번호	★ 추가	자동	연결된 PR Number. 없으면 빈 값
X	PR 상태	★ 추가	자동	open / merged / closed / N/A
Y	PR URL	★ 추가	자동	HYPERLINK
Z	자동 PR 생성	★ 추가	자동	success / failed / skipped (LLM Worker 가 기록)
AA	처리 소요(일)	★ 추가	수식	=IF(Q="", "", Q-T) 형태의 수식. 완료된 건만 일수 표시
AB	테스트 환경	★ 추가	자동	OS / 브라우저 / 디바이스 요약 (예: macOS / Chrome 124 / Wi-Fi)

열	컬럼명	구분	작성 주체	내용 / 비고
AC	최종 동기화	★ 추가	자동	이 행이 마지막으로 자동 업데이트된 시각

드롭다운 / 데이터 검증

- B, D, E, F, I, J, K, L, R, X 열은 데이터 검증으로 드롭다운을 설정한다 (Sheets API의 setDataValidation)
- 드롭다운 값이 변경되어도 자동화가 항상 정해진 목록 안에서만 값을 쓰므로 검증 위반이 발생하지 않는다

조건부 서식

- K 열 우선순위: P1=빨강, P2=주황, P3=노랑, P4=회색 (배경색)
- R 열 처리 상태: Open=빨강 글자, 진행중=주황 글자, 완료=초록 글자
- AA 열 처리 소요: 7 일 이상이면 빨강 배경

4.3 [Daily Snapshot] 시트

매일 17 시에 한 행씩 자동 추가되어 KPI의 시간 추이를 기록한다. Dashboard 시트의 추세 차트가 이 시트를 데이터 소스로 사용한다.

열	컬럼명	값 / 계산 방식
A	스냅샷 일자	스크립트 실행일 (YYYY-MM-DD)
B	총 이슈 누적	Raw Issues 시트의 행 수
C	오늘 신규	T 열(등록일시)이 당일인 행 수
D	Open 잔량	R 열이 '완료'가 아닌 행 수
E	P1 Open	K=P1 AND R<>완료
F	P2 Open	K=P2 AND R<>완료
G	P3 Open	K=P3 AND R<>완료
H	P4 Open	K=P4 AND R<>완료
I	오늘 완료	Q 열(처리일자)이 당일인 행 수
J	평균 처리 소요	AA 열의 평균 (완료된 건만)

열	컬럼명	값 / 계산 방식
K	자동 PR 시도	Z<>skipped 그리고 T 가 당일인 행 수
L	자동 PR 성공	Z=success 그리고 T 가 당일인 행 수
M	활성 테스터(7 일)	최근 7 일간 A 열의 unique 개수

4.4 [Dashboard] 시트

Raw Issues 와 Daily Snapshot 을 데이터 소스로 한 KPI + 차트 시트. 사람이 열어보는 화면이므로 한 화면 안에 들어가도록 배치한다. 모든 셀은 자동 갱신된다(외부 API 호출이 아니라 시트 함수와 차트의 자동 재계산 기능을 사용).

상단: KPI 카드 (6 개)

각 카드는 인접한 2~3 개 셀을 병합하여 큰 숫자를 보여준다. 셀 함수는 다음과 같이 구성한다.

KPI	위치	계산식 (Raw Issues 시트 기준)
오늘 신규 이슈	B2:C3 (병합)	=COUNTIF('Raw Issues'!T:T, ">="&TODAY())
Open 이슈 총수	D2:E3	=COUNTIF('Raw Issues'!R:R, "<>완료")-1 (-1 은 헤더 제외)
P1 Open	F2:G3	=COUNTIFS('Raw Issues'!K:K,"P1",'Raw Issues'!R:R,"<>완료")
평균 처리 소요(일)	H2:I3	=ROUND(AVERAGE('Raw Issues'!AA:AA), 1)
활성 테스터(7 일)	J2:K3	=SUMPRODUCT(1/COUNTIF(...))을 ARRAYFORMULA 로
자동 PR 성공률	L2:M3	=COUNTIF(Z:Z,"success")/COUNTIF(Z:Z,"<>skipped")

중단: 차트 4 개

- ① 일별 등록·완료 추세 라인 차트 → 데이터: Daily Snapshot 의 A, C, I 열 / 최근 14 일
- ② 원인 유형별 분포 도넛 차트 → 데이터: Raw Issues 의 L 열 피벗 / Open 건만

- ③ 테스트 영역별 Open 가로 막대 차트 → 데이터: Raw Issues 의 D 열 피벗 / Open 건만
- ④ 심각도 × 처리 상태 히트맵 → QUERY 함수로 K 열과 R 열의 크로스탭을 계산 후 조건부 서식으로 셀 배경색 적용

하단: 액션 리스트

- P1 + Open 목록: QUERY 함수로 추출. =QUERY('Raw Issues'!A:AC, "select U,D,H,N,T where K='P1' and R<>'완료' order by T asc")
- 7 일 이상 미처리: QUERY 함수로 추출. where TODAY()-T >= 7 AND R<>'완료'
- 이 목록은 행 수 제한이 없어 자동으로 늘어남

Looker Studio 연동 옵션

Google Sheets 내장 차트의 디자인이 부족하다고 느낄 경우, Looker Studio(구 Data Studio)를 Raw Issues / Daily Snapshot 시트에 연결하여 더 정교한 대시보드를 별도로 만들 수 있다. 무료이며, 시트 데이터 변경 시 자동 동기화된다. 본 1 차 릴리스에서는 Google Sheets 내장 차트로 충분하다고 판단한다.

4.5 Google Sheets 구현 가능성 검증

아래 표는 본 설계의 각 요소가 Google Sheets 에서 실제로 구현 가능한지를 정리한 것이다.

요구사항	가능 여부	구현 방식 / 참고
새 이슈 행 자동 추가	가능	spreadsheets.values.append API
특정 행의 일부 컬럼만 갱신	가능	Issue 번호(U 열)로 행 찾고 spreadsheets.values.batchUpdate 로 특정 셀만 부분 갱신
수작업 컬럼 보호	가능	자동화 코드가 L, M, N, O, P, S 컬럼을 절대 쓰지 않도록 한다 (코드 차원 보장). 추가로 시트 보호 기능으로 서비스 계정만 자동 컬럼을 쓸 수 있게 설정 가능
KPI 큰 숫자 표시	가능	셀 병합 + 큰 글꼴 + COUNTIF/AVERAGEIF 함수
추세 라인 차트	가능	Daily Snapshot 데이터를 소스로 한 LINE 차트
도넛 / 가로 막대 차트	가능	PIE / BAR 차트 (가로 막대는 BAR, 세로는 COLUMN)

요구사항	가능 여부	구현 방식 / 참고
히트맵 (심각도 × 상태)	가능	QUERY 함수로 크로스탭 계산 + 조건부 서식으로 셀 색상 그라데이션
드롭다운 셀 (데이터 검증)	가능	API 의 setDataValidation 또는 시트에서 직접 설정
조건부 서식 (배경색·글자색)	가능	AddConditionalFormatRuleRequest 로 자동 설정 또는 수동 설정
HYPERLINK (이슈/PR 클릭 이동)	가능	=HYPERLINK("https://github.com/...", "#142") 형태로 셀에 기록
필터 뷰 (P1+Open 등)	가능	필터 뷰 또는 Dashboard 시트에서 QUERY 로 별도 영역 표시
이미지 인라인 표시	제한적	=IMAGE() 함수로 가능하지만 GitHub 이미지는 인증이 필요해 동작하지 않는 경우 있음. 대신 URL 만 기록하는 것이 안전
드릴다운 (셀 클릭 시 상세 모달)	불가	Sheets 로는 불가. 필요하면 Looker Studio 또는 별도 웹 대시보드 필요
실시간 푸시 (Webhook 이 즉시 시트에 반영)	가능(주의)	API 호출은 즉시지만 사용자가 시트를 열고 있을 때만 시각적으로 반영됨. API 한도: 분당 60 건, 일 50 만 셀로 충분
컬럼별 권한 분리	불가	시트 단위 권한만 가능. 자동 컬럼 영역에 시트 보호를 걸어 서비스 계정만 편집 가능하게 설정

5. 기능별 상세 설계

5.1 [기능 1] 웹 폼 → GitHub Issue 등록

처리 흐름

12. 프론트엔드에서 필수 입력·이메일 형식·날짜 범위 검증 후 POST /api/issues 로 전송
13. Backend 는 GitHub App 인증으로 POST /repos/{owner}/{repo}/issues 호출
14. 성공 시 Issue 번호·URL 을 응답으로 받아 사용자에게 표시
15. 같은 트랜잭션 안에서 Sheet Syncer 에게 'issue_created' 이벤트 전달

Issue Body 마크다운 포맷

기존 템플릿 구조를 그대로 따른다. 신규 추가된 '테스트 환경'은 Summary 섹션 끝에 들여쓰기로 삽입.

Summary

- 테스트 계정 : tester01@example.com
- 접속 시간 : 26.05.19 16:05
- 테스트 영역 : Research agent
- 원인 유형 : Prompting
- 원인 상세 : 검색 결과 요약 누락
- 심각도 : 높음
- 테스트 환경 :
 - OS : macOS 14.5
 - 브라우저 : Chrome 124.0
 - 디바이스 : MacBook Pro 14
 - 네트워크 : Wi-Fi

테스트 시나리오 / 재현 방법 / 예상·실제 동작 / 문제 발생 화면

입력값·출력값 / 추가 의견

(기존 템플릿 그대로)

5.2 [기능 2] Open 이슈 → LLM 호출 → PR 자동 생성

트리거

- GitHub Webhook 이벤트 issues.opened 를 LLM Worker 가 수신
- 라벨이 beta-feedback 인 이슈만 처리 대상

LLM 호출 단계

16. 이슈 본문에서 Summary / 재현 방법 / 입력값·출력값 / 테스트 환경을 파싱
17. 관련 코드 후보 파일을 저장소에서 검색하여 함께 프롬프트로 제공

18. LLM API 호출: ① 원인 가설 ② 수정 후보 ③ diff 패치 초안
19. 응답 검증 후 새 브랜치 fix/issue-{번호}에 커밋
20. PR 생성: 본문에 'Fixes #{번호}' 포함하여 머지 시 이슈 자동 Close
21. 이슈 라벨 auto-pr-pending → auto-pr-created 교체, PR 링크를 이슈에 코멘트
22. Sheet Syncer 에게 'pr_created' 이벤트 전달 → Z, W, X, Y 컬럼 자동 채움

실패 처리

- LLM 응답이 적용 불가하면 PR 미생성, 이슈에 분석 결과만 코멘트
- auto-pr-failed 라벨 부여, Z 열에 failed 기록

5.3 [기능 3] Open PR 처리

- PR 처리는 수동 범위. 개발자가 코드 리뷰 후 머지/수정/반려
- PR 상태 변경 Webhook(pull_request.closed)을 Sheet Syncer 가 받아 X, Q, R, AA 컬럼 갱신

5.4 [기능 4] Google Sheet 동기화 (실시간 + 일일 17 시)

실시간 동기화 (Webhook 기반)

23. issues.opened: Raw Issues 에 새 행 추가 (A~K, T~AC, AB 채움; L~P 는 비움)
24. issues.edited / issues.labeled: 해당 행의 자동 컬럼만 갱신
25. issues.closed: R 열 = 완료, Q 열 = 머지 시점
26. pull_request.opened: 해당 이슈 행의 W, X, Y, Z 갱신
27. pull_request.closed (merged): 해당 행의 X = merged, R = 완료, Q = 오늘

일일 17:00 전수 동기화

28. Cron: 0 17 * * * (Asia/Seoul)
29. GitHub API 로 모든 이슈를 페이지네이션 조회 (state=all)
30. Raw Issues 시트와 비교하여 누락 행 추가, 불일치 행 갱신
31. Daily Snapshot 에 당일 통계 1 행 추가
32. 실행 결과(처리 건수, 오류 건수)를 Slack 에 알림

API 사용량 견적

- 일일 신규 이슈 50 건 가정 → append 50 회, batchUpdate 100 회 = 150 회 (분당 60 건 한도 여유)
- 전수 동기화 시 페이지 100 개 × append 5 회 = 500 회 (배치로 묶으면 50 회 수준)
- 일 50 만 셀 한도 대비 사용량은 5000 셀 미만으로 충분

6. 기술 스택

영역	권장 후보	선택 사유
Frontend	Next.js (React) + Tailwind	정적 호스팅, 폼 검증 용이
Backend	FastAPI (Python)	LLM/GitHub/Sheets SDK 모두 풍부
LLM API	Anthropic Claude API	코드 분석·diff 생성 품질
GitHub 연동	GitHub App + Webhook + REST API v3	권한 범위 좁고 레이트리밋 여유
Sheets 연동	Google Sheets API v4 + 서비스 계정	Python google-api-python-client 사용
스케줄러	GitHub Actions cron	별도 인프라 불필요, Secret 관리 통합
호스팅	Vercel + Cloud Run	Webhook 수신, 저비용

7. 일정 계획 (4 주)

주차	단계	주요 산출물	의존성 / 검증
W1	설계 & 셋업	- 요구사항 확정 - GitHub App 등록 - Google Sheet 템플릿 생성 (3 개 시트 + 기본 차트) - 서비스 계정 권한 부여 - 라벨 체계 정비	- 저장소 admin 권한 - LLM API 키 - Google Cloud 프로젝트
W2	기능 1+4 구현	- 웹 폼 UI (두 템플릿) - Backend → GitHub Issue 생성 - Sheet Syncer 실시간 부분 (Webhook) - 이미지 첨부 업로드	- 더미 저장소·시트 사전 검증 - 운영자가 시트 미리 검토
W3	기능 2 구현	- LLM Worker - 프롬프트 설계·튜닝 - 브랜치/커밋/PR 생성 - PR 상태 Sheet 동기화	- 과거 이슈 5 건 회귀 테스트
W4	일일 동기화 + 안정화	- 일일 17 시 Cron 작성 - Daily Snapshot append 로직 - Dashboard 시트 차트 4 개 + KPI 6 개 마무리 - 운영 매뉴얼 - 베타 오픈	- 시범 보고 3 일치 검증 - 롤백 절차

8. 리스크 및 대응

리스크	영향	대응
스팸/장난성 이슈 폭주	LLM 비용·노이즈	이메일 인증 / hCaptcha / 분당 제출 제한
LLM 이 잘못된 patch 생성	잘못된 PR 머지 위험	PR 은 반드시 사람 리뷰 후 머지
자동화가 시트 수작업 컬럼을 덮어씀	운영자 작업 손실	코드에서 L,M,N,O,P,S 컬럼 쓰기 차단. 시트 보호로 이중 안전
Google Sheets API 한도 초과	동기화 누락	batchUpdate 로 묶기, 지수백오프 재시도
동시 편집 충돌 (운영자 + 자동화)	데이터 일관성 깨짐	자동/수작업 컬럼 분리로 충돌 없음. 일일 동기화 시 자동 컬럼만 덮어씀
개인정보 유입(이메일 등)	Public 저장소 노출	저장소 Private 전환 검토. 시트 공유 권한 최소화
일일 보고 실패 누락	운영 가시성 저하	Slack/메일 별도 알림. 실패 시 자동 재시도 1 회

9. 성공 기준 (KPI)

- 이슈 제출 평균 소요 시간: 5 분 이하
- 이슈 등록 → 시트 동기화: 1 분 이내
- 이슈 등록 → PR 자동 생성: 5 분 이내
- 자동 PR 머지율: 30% 이상
- 일일 보고 누락률: 0% (월 1 회 이하 장애 허용)

10. 향후 확장

- Looker Studio 연동으로 더 정교한 대시보드 (드릴다운, 사용자 필터)
- 자동 생성 PR 에 대한 자동 테스트 실행 및 결과 코멘트
- Slack/Discord 봇으로 베타 테스터에게 처리 진행 상황 역방향 알림
- 이슈 중복 탐지 (임베딩 유사도 기반)
- 주간/월간 회고 리포트 자동 생성 (LLM 이 Daily Snapshot 데이터를 요약)